

MALWARE ANALYSIS REPORT

Beacon.exe

C2 Beacon | XOR & Base64 Encoding Analysis

Anthony Whitemann | April 19, 2026

Environment: Isolated VM Lab | Tools: IDA Free, WinHex, BinText, Wireshark

Classification: Personal Research

EXECUTIVE SUMMARY

Beacon.exe is a command-and-control (C2) beacon malware designed to establish and maintain covert communication with a remote attacker-controlled server. The sample targets www.practicalmalwareanalysis.com and was analyzed in a controlled, isolated virtual machine environment. The malware employs two distinct encoding schemes to evade detection: XOR encoding with a single-byte key to conceal its C2 domain within the binary's resource section, and Base64 encoding to obfuscate data transmitted in outbound HTTP GET requests.

Static analysis using BinText confirmed the complete absence of plaintext network indicators in the binary. Dynamic analysis via Wireshark captured live beacon traffic, and disassembly in IDA Free revealed the implementation details of both encoding routines. The findings demonstrate deliberate anti-analysis design intended to defeat signature-based detection tools and frustrate both automated and manual triage.

STATIC ANALYSIS

String Extraction

Initial triage with BinText produced no recognizable network indicators. The C2 domain is entirely absent from the binary's plaintext string table, a deliberate design choice that renders the sample invisible to automated string-based scanners and manual inspection alike.

Despite the absence of plaintext infrastructure, BinText surfaced the complete Base64 alphabet — ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/ — confirming that Base64 encoding is implemented within the binary. Additional high-entropy strings such as `K1ZXORXZwVZWLZI` and `ZUZWBHRH` are consistent with encoded data stored at rest within the executable.

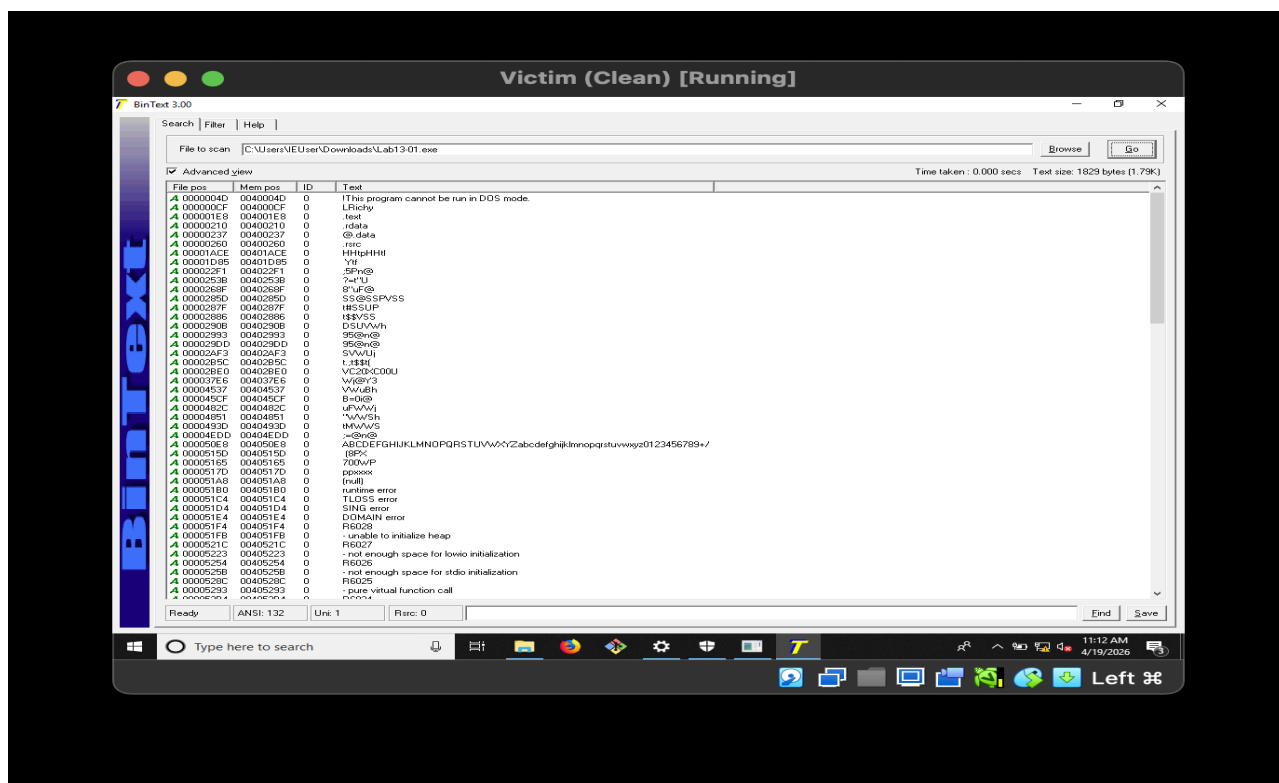


Figure 1 — BinText output. The Base64 alphabet is visible; no C2 domain appears in plaintext.

Resource Section Inspection

Examination of the binary's resource section using WinHex located an encoded byte sequence at file offset 0x7060. Applying a single-byte XOR with key 0x3B resolves the sequence to www.practicalmalwareanalysis.com. Placement in the resource section rather than the standard .data or .rdata segments reduces the likelihood that the domain will be flagged by automated PE analysis tools or antivirus heuristics during static scanning.

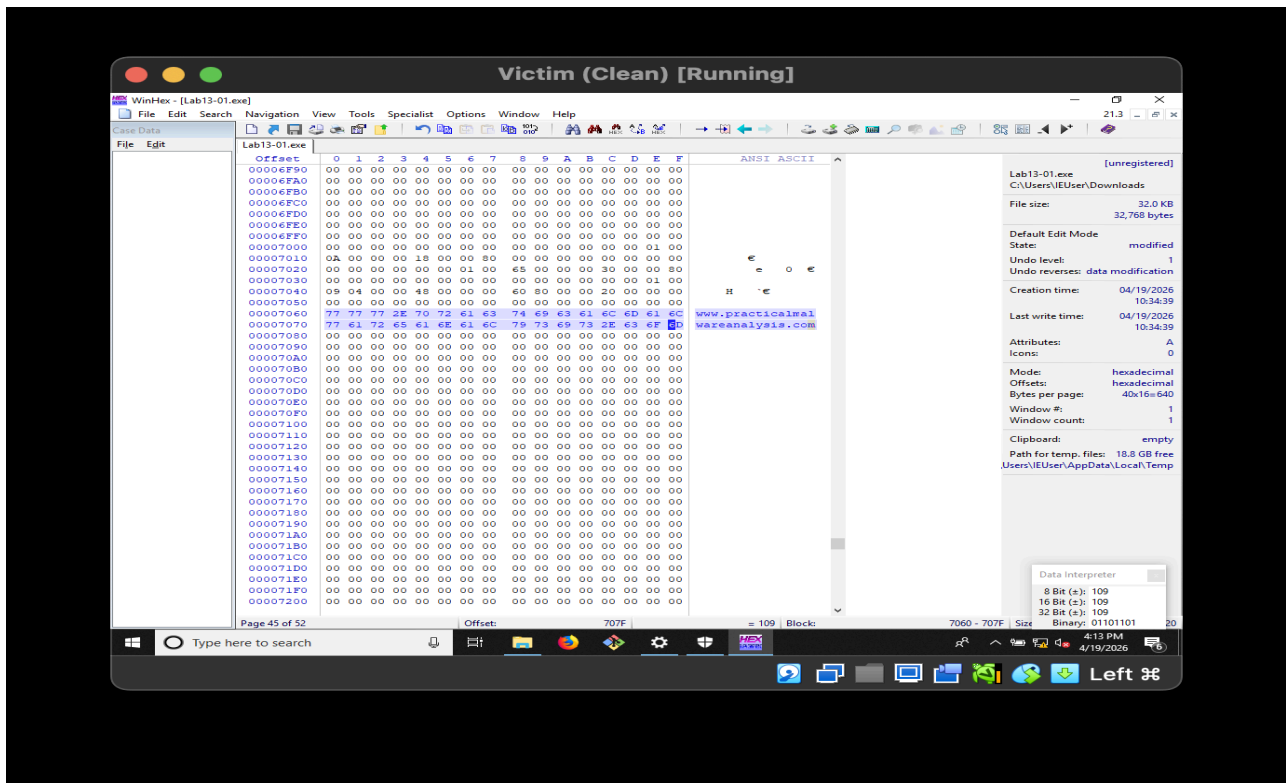


Figure 2 — WinHex view of the resource section. The XOR-encoded C2 domain is located at offset 0x7060.

ENCODING ANALYSIS

XOR Encoding — C2 Domain Obfuscation

The malware decodes its C2 domain at runtime using a simple XOR cipher keyed on 0x3B (59 decimal). The routine iterates over the resource bytes, XORs each byte against the key, and writes the plaintext result to a heap-allocated buffer. Because XOR with a fixed key is symmetric, the same function serves as both encoder and decoder. In IDA Free, the function was renamed `xorEncode_AnthonyWhiteman`; the cross-reference view confirmed a single call site, originating from the resource loading routine.

Windows API Call Chain

The resource loading sequence — `FindResourceA`, `LoadResource`, `GlobalAlloc`, `LockResource`, `SizeofResource` — is the standard pattern for reading PE resource data at runtime. Its combination with an immediately subsequent XOR loop and outbound socket activity is a reliable behavioral indicator of runtime decoding prior to C2 contact.

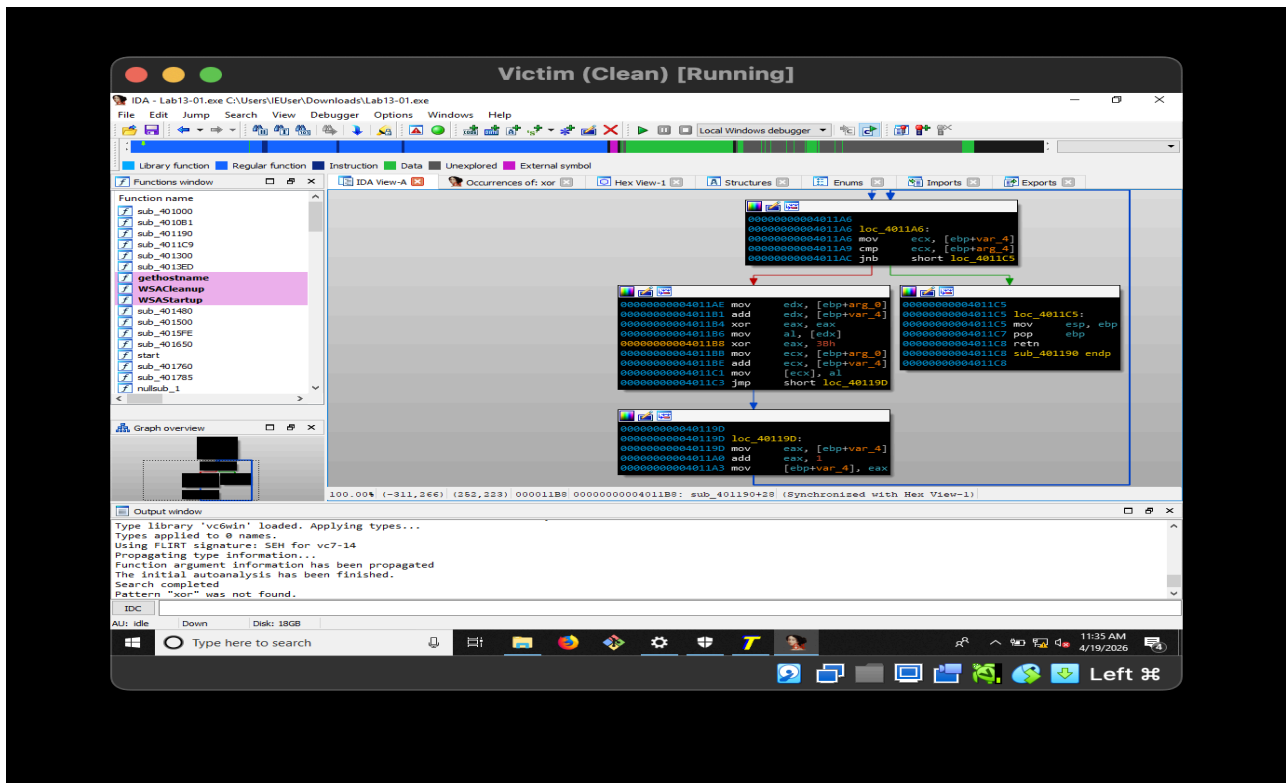


Figure 3 — IDA Free graph view of the XOR decode loop iterating over the resource section bytes.

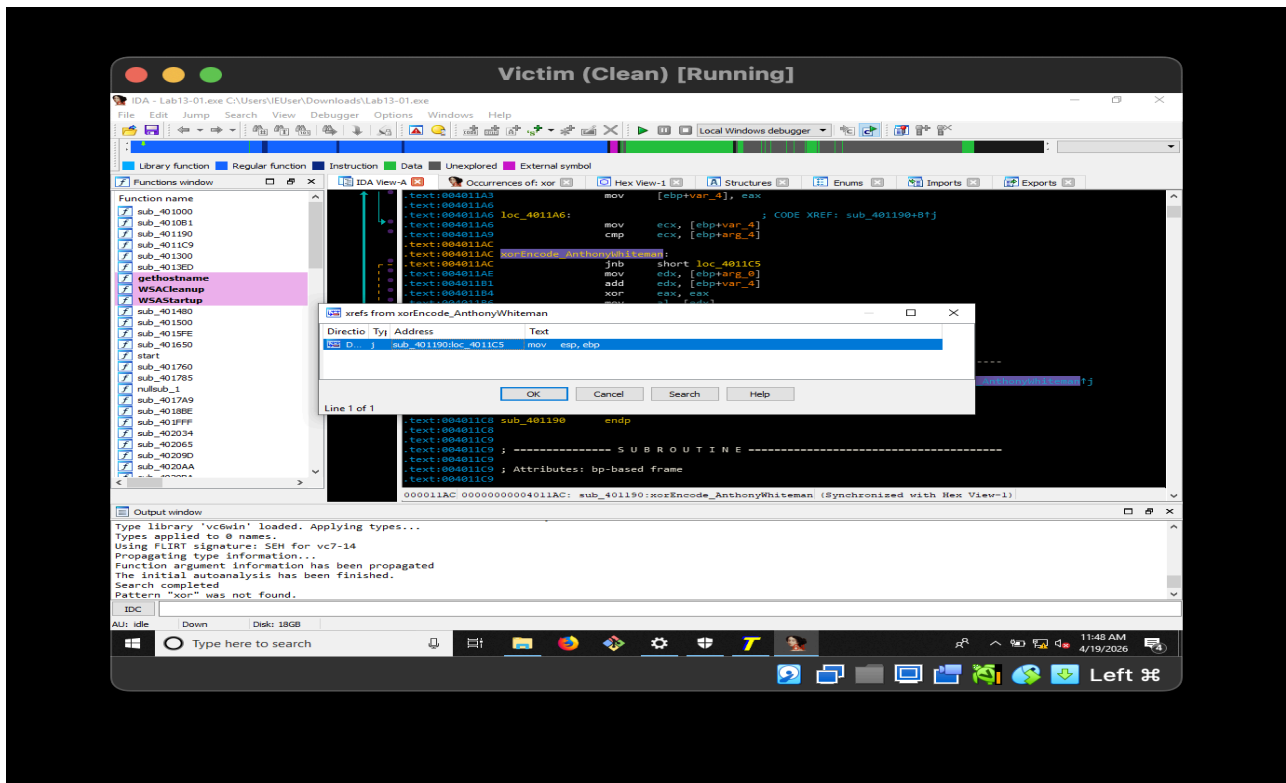


Figure 4 — IDA Free cross-reference view confirming a single call to the XOR routine from the resource loader.

Base64 Encoding — Network Traffic Obfuscation

Outbound beacon data is encoded using a custom Base64 implementation located at virtual address 0x00401000. The function references a standard Base64 alphabet table stored at 0x004050E8. Encoded output is embedded directly into the HTTP GET request path, producing strings such as `Vk1DVE1N` as observed in live Wireshark captures. Transmitting encoded data as a URL path segment keeps the traffic syntactically valid HTTP, making it harder to flag on protocol structure alone.

Base64 padding characters (= or ==) appear only when the input byte length is not a multiple of three. Because the encoded payload is derived from a fixed-length hostname, padding may not appear consistently. Analysts should not treat the absence of padding as evidence against Base64 being in use.

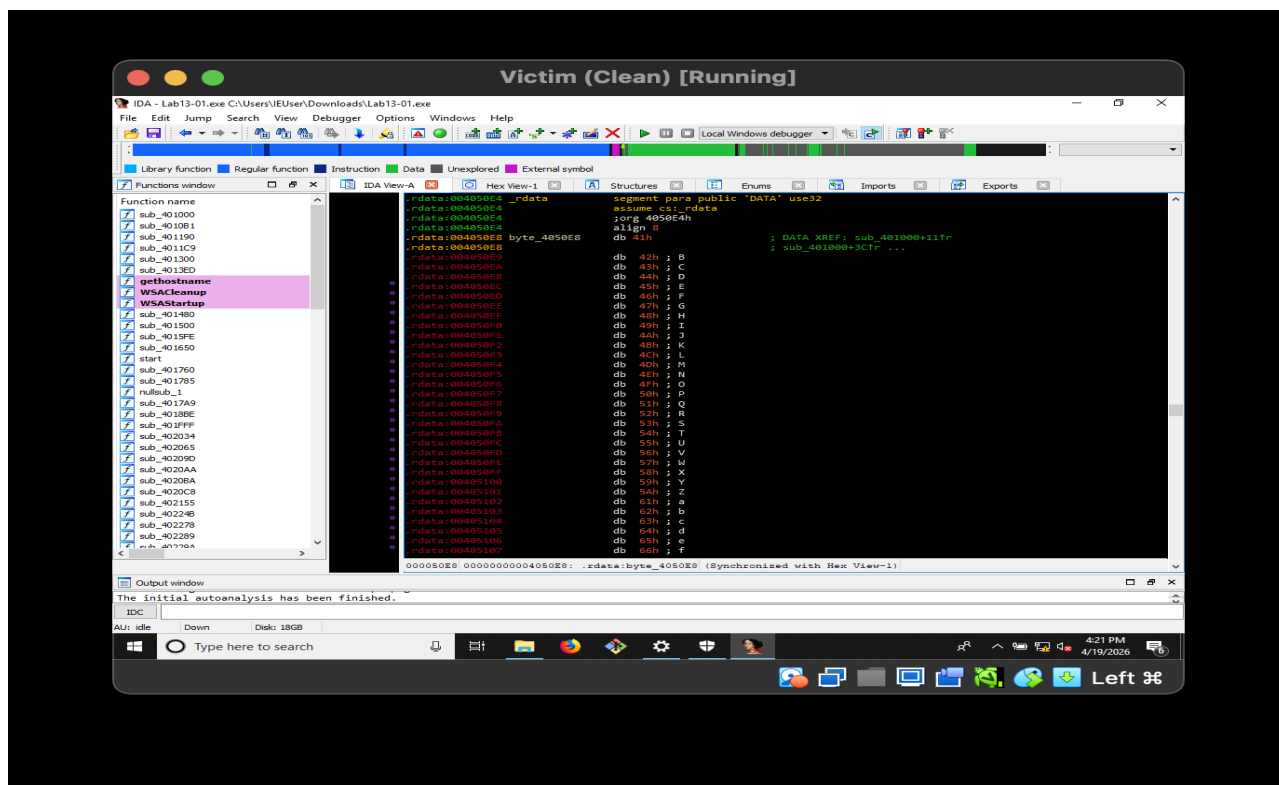


Figure 5 — IDA Free disassembly of the Base64 encoding function at 0x00401000.

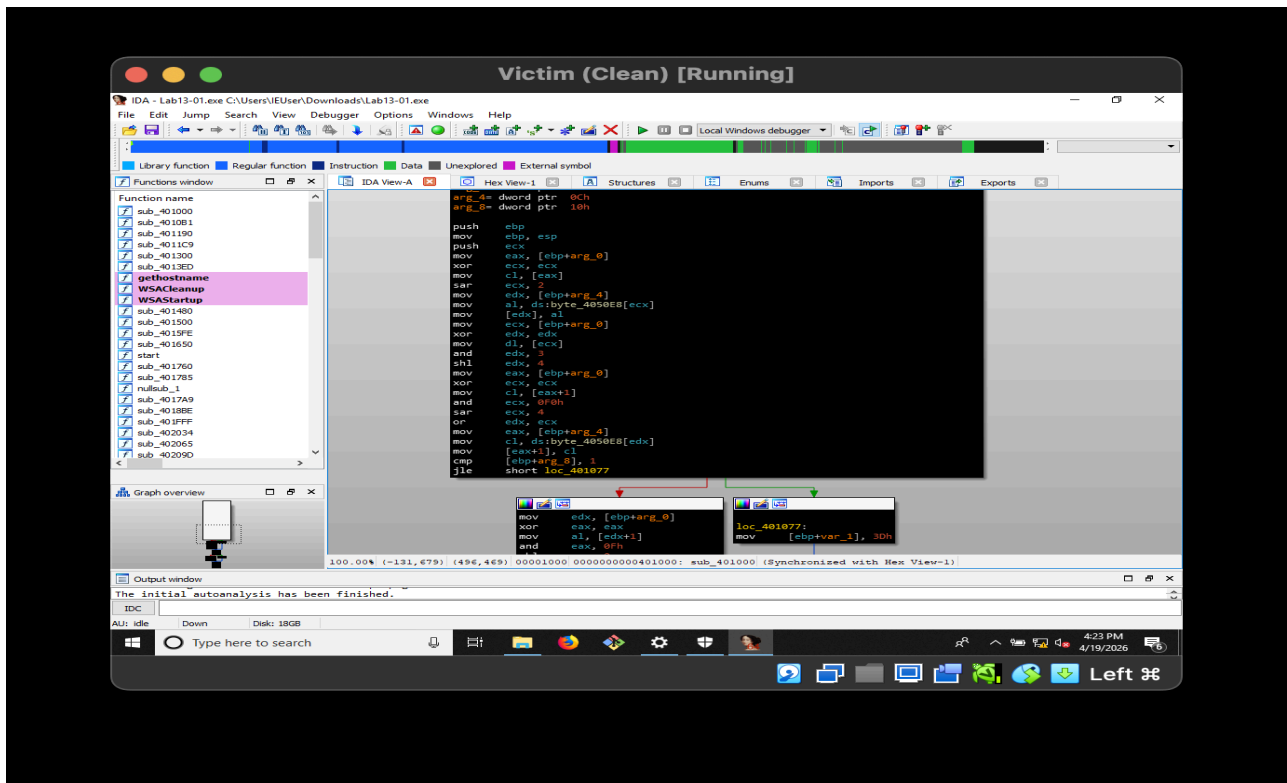


Figure 6 — IDA Free data section showing the Base64 alphabet table at 0x004050E8.

DYNAMIC ANALYSIS

Network Behavior

Executing the sample in an isolated VirtualBox environment — with INetSim providing simulated internet services — produced consistent, periodic HTTP GET requests to `www.practicalmalwareanalysis.com` on port 80. Each request carries a Base64-encoded string embedded in the URL path. The malware identifies itself with the User-Agent string `Mozilla/4.0`, impersonating a legacy Internet Explorer client to blend with standard enterprise web traffic.

The fixed beacon interval is characteristic of implants in a standby state, waiting for operator tasking. In a live environment the C2 server would respond with commands; INetSim returns its default HTML page, causing the malware to loop without taking further action. The regularity of the interval is itself a detectable pattern — identical GET requests at consistent timing to the same host are a strong indicator of beaconing behavior.

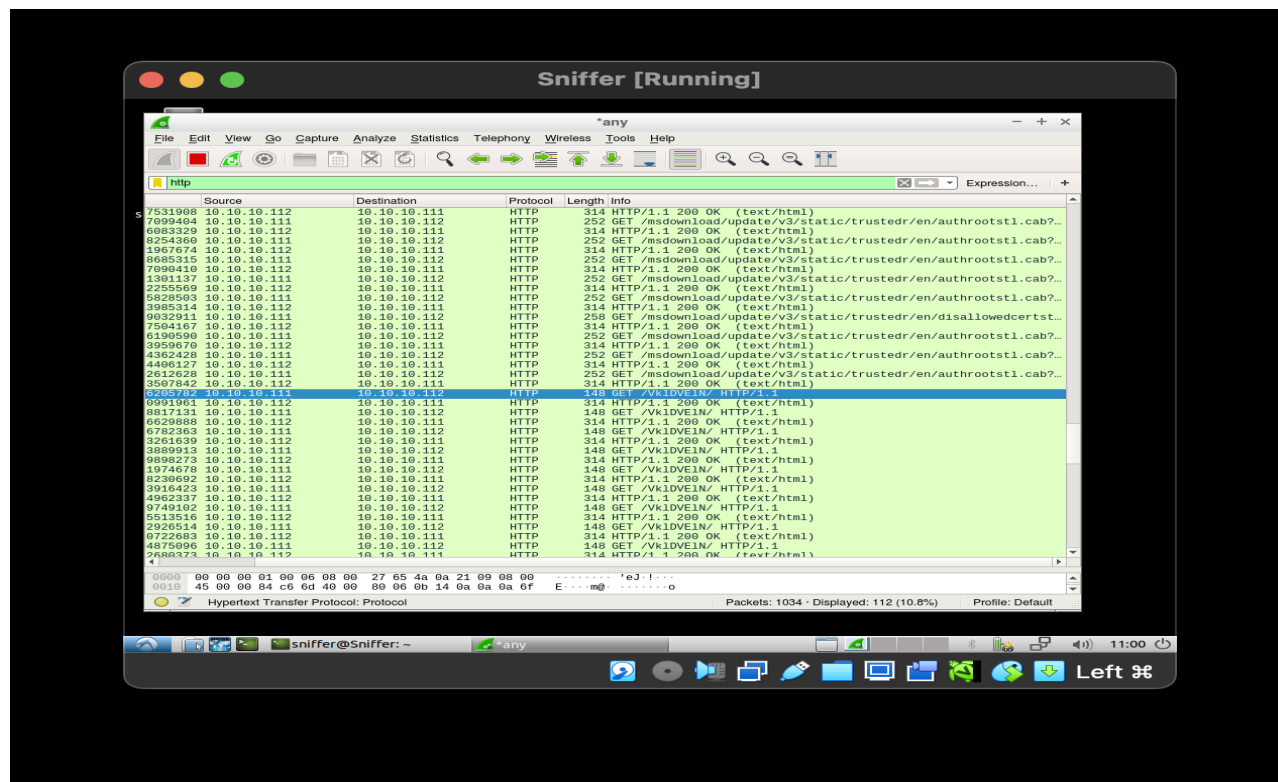


Figure 7 — Wireshark capture showing periodic HTTP GET beacons. The Base64-encoded path segment is visible in each request.

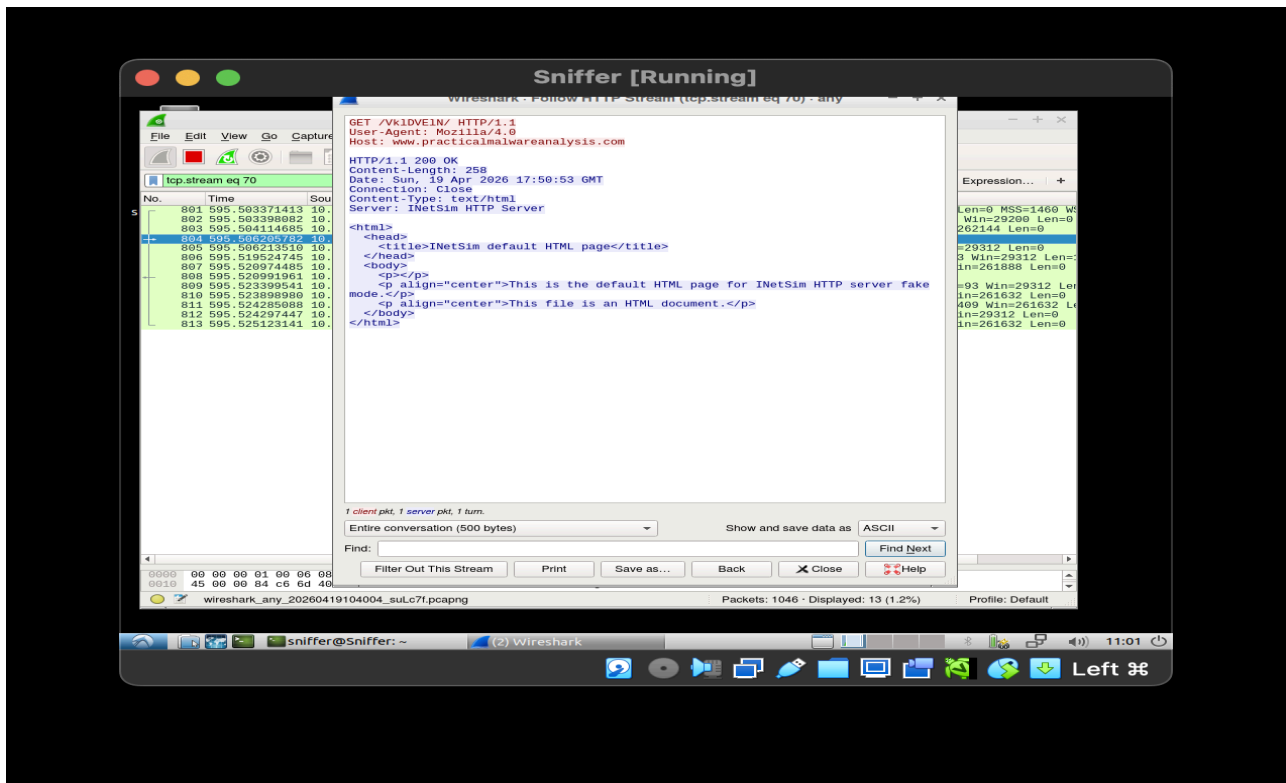


Figure 8 — Wireshark Follow HTTP Stream view showing the complete request and INetSim default response.

INDICATORS OF COMPROMISE

INDICATOR	TYPE	CONTEXT
www.practicalmalwareanalysis.com	Domain	C2 server address
0x3B (59 decimal)	XOR Key	Single-byte resource decode key
0x00007060	File Offset	Encoded C2 domain in resource section
0x00401000	Function VA	Base64 encoding routine
0x004050E8	Data VA	Base64 alphabet table
Mozilla/4.0	User-Agent	HTTP traffic masquerading string
Vk1DVE1N	URL Pattern	Sample Base64 beacon path segment
FindResourceA / LoadResource	API Pattern	Resource section access at runtime
GlobalAlloc / LockResource	API Pattern	Runtime decode buffer allocation

RECOMMENDATIONS

Network Controls

Block outbound HTTP to uncategorized or unverified external domains at the network perimeter. Any domain not present on an approved allow-list should be denied by default, forcing beacon traffic to fail before it reaches the C2 server. Separately, deploy detection logic for periodic GET requests containing high-entropy path segments — Base64-encoded data in URL paths is uncommon in legitimate enterprise traffic and is detectable using entropy scoring or character-set analysis in a SIEM or IDS.

Endpoint Controls

Apply behavioral detection rules for the API call chain associated with runtime resource decoding: any process that executes FindResourceA, LoadResource, and GlobalAlloc in close succession and subsequently initiates outbound socket activity should be flagged for review. This pattern is rare in benign software and provides a reliable host-based indicator independent of signature matching.

Incident Response

For a confirmed infected host: isolate the machine immediately at the network level, terminate the malicious process, and remove the executable. Restore from a verified clean snapshot rather than attempting in-place remediation. Prior to restoration, acquire a full memory image for offline analysis to confirm that no secondary payloads were staged, injected into other processes, or persisted through registry or scheduled task mechanisms during the infection window.